Many JWT libraries have built-in support for retrieving keys from a JWK Set over HTTPS, including periodically refreshing them. For example, the Nimbus JWT library that you used in chapter 6 supports retrieving keys from a JWK Set URI using the `RemoteJWKSet` class:

```
var jwkSetUri =
URI.create("https://as.example.com:8443/jwks_uri");
var jwkSet = new RemoteJWKSet(jwkSetUri);
```

Listing 7.11 shows the configuration of a new `SignedJwtAccessTokenStore` that will validate an access token as a signed JWT. The constructor takes a URI for the endpoint on the AS to retrieve the JWK Set from and constructs a `RemoteJWKSet` based on this. It also takes in the expected issuer and audience values of the JWT, and the JWS signature algorithm that will be used. As you'll recall from chapter 6, there are attacks on JWT verification if the wrong algorithm is used, so you should always strictly validate that the algorithm header has an expected value. Open the src/main/java/com/manning/apisecurityinaction/token folder and create a new file SignedJwtAccessTokenStore.java with the contents of listing 7.11. You'll fill in the details of the `read` method shortly.

> **TIP** If the AS supports discovery (see section 7.2.3) then it may advertise its JWK Set URI as the `jwks_uri` field of the discovery document.

Listing 7.11 The SignedJwtAccessTokenStore